

SRS

Software Requirements Specification (SRS)

Blockchain Voting System

Cryptographic E-Voting & Ledger Module

Document Version: 1.0
Prepared By: Student Project Team (TYBSc CS / AI-ML)
Institution: Thakur Ramnarayan College of Arts & Commerce (TRCAC)
Date: July 2026
Status: Final Draft

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. User Roles & Characteristics
- 4. Functional Requirements
- 5. Non-Functional Requirements
- 6. System Constraints
- 7. External Interface Requirements
- 8. Data Requirements

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Blockchain Voting System — E-Voting Module**. It is intended as a reference for the student development team, instructors, and any stakeholders involved in reviewing, testing, or extending the system.

1.2 Scope

The **Blockchain Voting System** is a web-based platform designed to streamline secure and tamper-evident election processes at Thakur Ramnarayan College of Arts & Commerce. This document covers the decentralized voting portion of the system, which manages elections across degree programs:

Program	Classes
B.Sc (Computer Science)	FYBScCS, SYBScCS, TYBScCS
B.Sc (Information Technology)	FYBScIT, SYBScIT, TYBScIT
Bachelor of Arts	FYBA, SYBA, TYBA
BAMMC	FYBAMMC, SYBAMMC, TYBAMMC
Bachelor of Management Studies	FYBMS, SYBMS, TYBMS
Bachelor of Commerce	FYBCom, SYBCom, TYBCom
Bachelor of Accounting & Finance	FYBAF, SYBAF, TYBAF

The system has three user roles: **Admin**, **Voter**, and **Observer**, each with distinct dashboards and privileges.

1.3 Definitions & Acronyms

Term	Definition
BVS	Blockchain Voting System
TRCAC	Thakur Ramnarayan College of Arts & Commerce
PoW	Proof-of-Work mining algorithm
ECDSA	Elliptic Curve Digital Signature Algorithm (SECP256R1)
JWT	JSON Web Token — used for session authentication
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
AES-256	Advanced Encryption Standard with 256-bit key

1.4 References

- Project Codebase: blockchain-voting-system (GitHub Repository)
- Security Documentation: SECURITY_SPECS.md
- Database Schema Documentation: DATABASE_SCHEMA.md
- System Architecture Documentation: SYSTEM_ARCHITECTURE.md

1.5 Overview

The Blockchain Voting System is divided into two main logical layers:

1. **Frontend** — A React + Vite Single Page Application (SPA) with Tailwind CSS.
2. **Backend** — A FastAPI (Python 3.12) REST API connected to SQLite / PostgreSQL database and custom Python Blockchain Engine.

2. Overall Description

2.1 Product Perspective

The Blockchain Voting System is a standalone web application that replaces manual, paper-based voting processes. The system is intended to be used by students, faculty, and administrators of TRCAC.

The system integrates with:

- **SQLite / PostgreSQL** — relational database for app data
- **Custom Python Blockchain** — in-memory SHA-256 ledger
- **FastAPI WebSockets** — real-time vote push updates

2.2 Product Functions (High-Level)

1. User Account Management — Admin creates voter and observer accounts; role-based login
2. Election Management — Admin creates, schedules, activates, and closes elections
3. Candidate Management — Admin creates candidate profiles with manifesto details
4. Bulk Voter Enrollment — Admin bulk-enrolls voters via CSV upload
5. 1-Click Encrypted Voting — Voter casts single vote encrypted with AES-256 GCM
6. Digital Signature Signing — Transactions signed using single-use voter ECDSA key pairs
7. Proof-of-Work Block Mining — Transactions packaged into blocks with SHA-256 difficulty targets
8. Downloadable QR Receipt — Generates printable receipt containing receipt hash and block index
9. Blockchain Explorer — Public block browser, transaction search, and 1-click ledger auditor
10. AI Fraud Radar — Analyzes vote velocity bursts (<2s), duplicate IP clusters, and threat scores
11. Live Transparency Dashboard — Observer views live Recharts standings and vote percentages

2.3 User Classes

User Class	Description
Admin	Full control of the system. Creates accounts, manages elections, candidate profiles, and bulk voter data.
Voter	Casts votes in active elections, generates QR receipts, and verifies vote hash on blockchain ledger.
Observer	Read-only access. Watches live results, inspects block explorer, verifies chain integrity, and views AI fraud alerts.

2.4 Operating Environment

- **Client Side:** Modern web browser (Chrome, Firefox, Edge, Safari)
- **Server Side:** Python 3.12+, FastAPI 0.111+, Uvicorn 0.30+
- **Database:** SQLite (local dev) / PostgreSQL (production)
- **Frontend Build Tool:** Vite 5.x
- **Deployment:** Docker, Docker Compose, GitHub Actions CI/CD

2.5 Design and Implementation Constraints

- Frontend is built with **React 19** and **Tailwind CSS 3.4**
- Backend uses Python 3.12 with FastAPI REST framework
- JWT tokens expire in **24 hours** for session security

- Single-vote policy strictly enforced per voter per election
- CORS is strictly configured to allow only registered frontend URL

2.6 Assumptions and Dependencies

- Users have stable internet connectivity.
- Every student possesses a valid institutional email.
- Custom Python Blockchain ledger service is operational.

3. User Roles & Characteristics

3.1 Admin

- **Technical proficiency:** Moderate; expected to use a web-based admin dashboard
- **Access level:** Full system access
- **Typical tasks:** Create voter accounts, manage elections, bulk voter enrollment, view audit logs
- **Authentication:** Stored in User table (role = 'admin')

3.2 Voter

- **Technical proficiency:** Basic to moderate
- **Access level:** Scoped to assigned active elections
- **Typical tasks:** View candidates, cast encrypted vote, download QR receipt, verify hash
- **Authentication:** Stored in User table (role = 'voter')

3.3 Observer

- **Technical proficiency:** Basic to moderate
- **Access level:** Read-only access to public transparency metrics
- **Typical tasks:** View live results, inspect block ledger, audit chain, view AI fraud alerts
- **Authentication:** Stored in User table (role = 'observer')

4. Functional Requirements

4.1 Authentication & Authorization Module

FR-AUTH-01: User Login

- **Description:** The system shall allow admin, voters, and observers to log in using email/username and password.
- **Input:** Email/Username, Password
- **Process:** System searches user record and verifies password using bcrypt. Returns JWT token.
- **Output:** JWT access token + user profile data
- **Token Expiry:** 24 hours
- **Error Cases:** Invalid credentials -> HTTP 400

FR-AUTH-02: Admin User Creation

- **Description:** Only authenticated admins can create new user accounts.
- **Input:** Username, email, password, role (voter/observer)
- **Process:** Admin endpoint creates user record.
- **Output:** New user account created
- **Error Cases:** Duplicate email -> HTTP 400

FR-AUTH-03: Logout

- **Description:** Users can log out, clearing local storage JWT token.
- **Input:** Logout action
- **Process:** Token removed from client storage.
- **Output:** User redirected to /login

4.2 Admin Dashboard Module

FR-ADMIN-01: Dashboard Overview

- **Description:** Displays total elections, active elections, registered voters, total votes, blockchain health.
- **Route:** GET /api/v1/admin/metrics

FR-ADMIN-02: Election Management

- **Description:** Admin can create, edit, activate, and close elections.
- **Route:** POST /api/v1/elections, PUT /api/v1/elections/:id/status

FR-ADMIN-03: Candidate Studio

- **Description:** Admin can register candidate profiles with name, party, manifesto, and avatar URL.
- **Route:** POST /api/v1/elections/:id/candidates

FR-ADMIN-04: Bulk Voter Enrollment

- **Description:** Admin can upload a CSV file to bulk-create voter accounts.
- **Route:** POST /api/v1/admin/import-voters-csv

FR-ADMIN-05: Audit Log View

- **Description:** Admin can view immutable audit logs of all user actions.
- **Route:** GET /api/v1/admin/audit-logs

4.3 Voter Dashboard Module

FR-VOTER-01: Active Elections List

- **Description:** Voters can view all currently active elections and candidate profiles.
- **Route:** GET /api/v1/elections

FR-VOTER-02: 1-Click Encrypted Vote

- **Description:** Voter casts vote encrypted with AES-256 GCM and signed with ECDSA key pair.
- **Route:** POST /api/v1/votes/cast

FR-VOTER-03: Anti-Double Voting

- **Description:** Enforces single vote per user per election.
- **Route:** POST /api/v1/votes/cast (Checks existing vote)

FR-VOTER-04: Downloadable QR Receipt

- **Description:** Displays downloadable QR receipt containing receipt hash and block index.
- **Route:** GET /api/v1/votes/my-receipts

FR-VOTER-05: Receipt Verification

- **Description:** Voters can verify any receipt hash against blockchain ledger.
- **Route:** GET /api/v1/votes/verify-receipt/:hash

4.4 Custom Blockchain Module

FR-CHAIN-01: Block Mining

- **Description:** Packages queued transactions into blocks meeting SHA-256 Proof-of-Work difficulty target.
- **Route:** Internal Blockchain Engine

FR-CHAIN-02: Merkle Tree Hash

- **Description:** Header contains binary Merkle root hash calculated from transaction hashes.
- **Route:** Internal Merkle Tree Generator

FR-CHAIN-03: Ledger Explorer

- **Description:** Public API allowing block browsing, transaction lookup, and 1-click chain audit.
- **Route:** GET /api/v1/blockchain/blocks, GET /api/v1/blockchain/verify-chain

4.5 AI Fraud Radar Module

FR-AI-01: Fraud Risk Index

- **Description:** AI engine analyzes vote velocity bursts (<2s), duplicate IP clusters, and double-vote logs to output a threat index (0-100%).
- **Route:** GET /api/v1/ai/fraud-analysis/:election_id

5. Non-Functional Requirements

5.1 Performance

Metric	Requirement
--------	-------------

API Response Time	< 500 milliseconds for REST endpoints under normal load
Block Mining	PoW block mined within 2 seconds for difficulty target = 2
Real-Time Push	WebSocket vote updates broadcast within 1 second
Concurrent Users	Supports at least 100 concurrent voters (student project scale)

5.2 Security

Requirement	Detail
Authentication	JWT Bearer tokens, 24-hour expiry
Password Storage	bcrypt password hashing with salt
Vote Confidentiality	AES-256 GCM symmetric vote payload encryption
Transaction Signing	ECDSA SECP256R1 asymmetric key pair digital signatures
CORS Protection	Whitelist-only: localhost:3000 and Vercel URL
Rate Limiting	Strict route limiters on login and vote casting endpoints

5.3 Reliability & Usability

- **Reliability:** System maintains 99.9% uptime. Immutable blockchain ledger guarantees zero vote deletion or modification.
- **Usability:** Responsive glassmorphic layout built with Tailwind CSS, supporting dark and light mode toggle.
- **Scalability:** Stateless JWT design allows backend scaling; relational DB supports SQLite and PostgreSQL.

6. System Constraints

- 1. Proof-of-Work difficulty target is set to 2 zero-prefix hashes for MVP efficiency.
- 2. Custom Python Blockchain operates in-memory synced to SQLite database.
- 3. Single vote policy strictly enforced per voter per election.

7. External Interface Requirements

7.1 User Interfaces

Page	Role	Description
Login / Register	Public	Authentication portal with role selector
Admin Dashboard	Admin	Overview metrics, elections CRUD, CSV import, audit logs
Voter Dashboard	Voter	Active elections, 1-click vote casting, QR receipts
Observer Dashboard	Observer	Live Recharts standings, block explorer, AI fraud radar
Blockchain Explorer	Public	Mined blocks, transaction search, 1-click audit

7.2 Software & Communication Interfaces

- **Protocol:** HTTPS (production), HTTP (local development)
- **Data Format:** JSON (REST API & WebSockets)
- **Auth Header:** Authorization: Bearer <JWT_TOKEN>

8. Data Requirements

8.1 Data Entities

Entity	Table	Key Fields
User	users	id, email, username, hashed_password, role, is_active
Election	elections	id, title, description, status, start_time, end_time
Candidate	candidates	id, election_id, name, party, manifesto, vote_count
Vote	votes	id, user_id, election_id, candidate_id, voter_hash, tx_hash, block_index, receipt_hash
Block	blocks	id, index, timestamp, previous_hash, hash, nonce, merkle_root, signature
Transaction	transactions	id, tx_hash, block_index, election_id, voter_hash, encrypted_vote, signature
AuditLog	audit_logs	id, user_id, user_email, action, details, ip_address, timestamp

UML

UML Diagrams — Complete Set

Blockchain Voting System

Table of Contents

1. Use Case Diagrams (Admin, Voter, Observer)
2. Class Diagrams (User & Auth, Content & Block, Assessment & Vote)
3. Sequence Diagram — Login Flow
4. Sequence Diagram — Encrypted Vote & Block Mining Flow
5. Sequence Diagram — Receipt Verification Flow
6. Activity Diagram — AI Fraud Detection & Velocity Marking
7. Component Diagram
8. Entity-Relationship (ER) Diagram

1. Use Case Diagrams

1.1 Admin Use Cases

Admin actor interacts with: Login/Logout, Create Election, Add Candidate, Bulk Enroll Voters CSV, View Dashboard Stats, View Audit Logs, Check Blockchain Health.

1.2 Voter Use Cases

Voter actor interacts with: Login/Logout, View Active Elections, View Candidates, Cast Encrypted Vote, View QR Receipt, Verify Receipt Hash.

1.3 Observer Use Cases

Observer actor interacts with: View Live Recharts Standings, Inspect Mined Blocks, Search Transaction, Trigger Chain Audit, View AI Fraud Alerts.

2. Class Diagrams

2.1 User & Auth Classes

User Base Class: +id, +email, +username, +hashed_password, +role

Subclasses / Roles: AdminUser, VoterUser, ObserverUser.

2.2 Content & Block Classes

Block Class: +index, +timestamp, +previous_hash, +hash, +nonce, +merkle_root, +signature, +calculate_hash()

Blockchain Engine Class: +chain[], +pending_txs[], +mine_pending_transactions(), +is_chain_valid().

3. Sequence Diagrams

3.1 Login Flow

Voter -> React Frontend -> POST /api/v1/auth/login -> FastAPI API -> SQLAlchemy DB (verify bcrypt) -> Return JWT Token.

3.2 Encrypted Vote & Block Mining Flow

Voter -> React Frontend -> AES-256 Encrypt -> ECDSA Sign -> POST /api/v1/votes/cast -> FastAPI -> Blockchain Engine (PoW Mine Block) -> DB Sync -> WebSocket Broadcast.

ADD

Architecture Design Document (ADD)

Blockchain Voting System

Version: 1.0 | Date: July 2026 | Scope: Full-Stack Decoupled Architecture

1. Introduction & Architectural Goals

Goal	Description
Separation of Concerns	UI never touches database directly — frontend React SPA and backend FastAPI REST API are fully decoupled.
Role-Based Security	Every protected endpoint enforces strict Admin / Voter / Observer role checks via JWT claims.
Stateless Auth	JWT-based authentication allows horizontal scaling across multiple backend worker instances.
Tamper Immutability	Votes are recorded in SHA-256 blocks with Merkle roots and Proof-of-Work headers.
Vote Secrecy	Candidate choices encrypted with AES-256 GCM prior to block inclusion.

2. System Context & External Systems

Actor / System	Type	Role
Admin	User	Creates elections, candidate profiles, bulk voter enrollment, monitors metrics.
Voter	User	Casts encrypted vote, views active elections, generates QR receipt.
Observer	User	Monitors live Recharts standings, inspects block explorer, checks AI alerts.
SQLite / PostgreSQL	Database	Relational data store for user profiles, elections, and blocks.
Python Blockchain	Core Engine	SHA-256 PoW miner, Merkle tree generator, ECDSA signature verifier.
Vercel / Docker	Deployment	Static CDN frontend hosting & containerized ASGI backend runtime.

3. Frontend Architecture (React 19 + Vite)

Technology	Version	Role
React	19.0	UI Component Library
Vite	5.x	Build tool & HMR dev server
React Router DOM	6.x	Client-side declarative routing
Tailwind CSS	3.4	Utility-first styling & Dark Mode
Framer Motion	11.x	Smooth page animations
Recharts	2.x	Live vote distribution charts
Axios	1.x	HTTP client with JWT interceptors

4. Backend Architecture (FastAPI + Python 3.12)

Technology	Version	Role
Python	3.12+	Server runtime environment
FastAPI	0.111+	High-performance ASGI web framework
SQLAlchemy	2.0+	Relational ORM database mapping
PyJWT	2.8+	JSON Web Token signing & decoding
bcrypt	4.1+	Salted password hashing
Cryptography	42.0+	ECDSA SECP256R1 & AES-256 GCM algorithms
WebSockets	12.0+	Real-time bidirectional event manager

4.1 API Route Map

Prefix	File	Key Endpoints
/api/v1/auth	auth.py	POST /login, POST /register, GET /me
/api/v1/elections	elections.py	GET /, POST /, PUT /:id/status, POST /:id/candidates
/api/v1/votes	votes.py	POST /cast, GET /my-receipts, GET /verify-receipt/:hash
/api/v1/blockchain	blockchain.py	GET /blocks, GET /status, GET /verify-chain
/api/v1/admin	admin.py	GET /metrics, POST /import-voters-csv, GET /audit-logs
/api/v1/ai	ai.py	GET /fraud-analysis/:election_id
/api/v1/observer	observer.py	GET /live-results/:election_id

5. Key Design Decisions Matrix

#	Decision	Choice	Rationale	Tradeoff
1	User storage	Single users table with role enum	Unified auth lookup without multi-table queries.	Requires role checks on every route.
2	Vote Secrecy	AES-256 GCM payload encryption	Prevents vote choices from showing plaintext on block explorer.	Requires server encryption key.
3	Voter Auth	ECDSA SECP256R1 Key Pairs	Digital signature guarantees vote cannot be forged.	Keypair generated per vote.
4	Real-Time	FastAPI WebSockets	Instant push broadcasts to live dashboards.	Active socket connection management.
5	AI Threat Radar	Heuristic Velocity Analysis	Detects bursts (<2s), duplicate IPs, double voting.	Thresholds require tuning.